

HBDT-SumPath | CKKS | SEAL

DT

POC - Inference Privée sur Arbres de Decision

HBDT-SumPath | Approximation Polynomiale Adaptive | CKKS

My_POC_All - Rapport Technique Complet

TABLE DES MATIERES

1. Presentation generale du POC	3
2. Architecture et structure du projet	4
3. Pipeline d'inference - vue d'ensemble	5
4. Module d'approximation polynomiale (SoftStepApprox)	6
5. Mode Soft Adaptatif - algorithme et analyse de precision	8
6. Chiffrement homomorphe CKKS - parametres et analyse	10
7. Algorithme HBDT-SumPath et DataLayout	13
8. Resultats experimentaux complets	15
9. Analyse comparative : precision adaptative vs. precision HE	20
10. Limitations et cas d'echec observes	22
11. Conclusion	23

1 - PRESENTATION GENERALE DU POC

Le projet **My_POC_All** est un démonstrateur complet d'**inference privée sur arbres de decision**. Il combine trois briques algorithmiques : (1) l'**approximation polynomiale adaptative** de la comparaison binaire, (2) l'algorithme **HBDT-SumPath** (Shin et al.) pour structurer l'inference dans les slots d'un chiffretexte CKKS, et (3) le chiffrement homomorphe **CKKS via Microsoft SEAL v4.1.2**. L'objectif est de permettre à un serveur d'évaluer un arbre de decision sur des données client chiffrées, sans jamais voir les données en clair.

1.1 Objectifs du POC

- Valider la faisabilité d'une inference homomorphe complete sur des jeux de données réels.
- Comparer les modes d'inference : **Hard** (seuil exact), **Soft global** (polynôme uniforme), **Soft adaptatif** (degré par nœud selon le min_gap).
- Mesurer l'impact du choix du degré polynomial sur la précision, en clair et sous CKKS.
- Évaluer les temps de chiffrement, d'inference et de déchiffrement sur des datasets variés.
- Identifier les limites mémoire et de profondeur multiplicative du schéma CKKS.

1.2 Datasets utilisés

Dataset	Features	Classes	Echantillons test	Precision arbre / dernier run clair
Iris	4	3	50	100.00 % (run 20 samples)
Cancer	30	2	188	100.00 % (soft adaptatif normalise, run 32 samples)
Wine	13	3	59	100.00 % (run 26 samples)
Digits	64	10	1797	100.00 % (run 32 samples)
Breast	30	2	86	90.62 % (soft adaptatif normalise, run 32 samples)
Heart	13	2	45	84.62 % (soft adaptatif normalise, run 26 samples)
Steel	33	2	292	100.00 % (soft adaptatif normalise, run 32 samples)
Spam	57	2	691	100.00 % (hard clair, run 2 samples ; HE OOM)
Spam2	57	2	691	100.00 % (hard clair, run 2 samples ; HE non valide)

Table 1 - Jeux de données utilisés et précision de l'arbre de decision en clair.

Note : Les datasets *Breast*, *Heart* et *Steel* utilisent des arbres pré-entraînés fournis par le projet *SortingHat* (Rust/C++ PDTE), dont la précision intrinsèque est limitée indépendamment de l'approximation polynomiale.

2 - ARCHITECTURE ET STRUCTURE DU PROJET

Le projet est organisé en modules C++17 compilés via CMake et exécutés dans un conteneur Podman (Ubuntu 22.04) pour garantir la reproductibilité. Un script Python **train_and_export.py** gère l'entraînement sklearn et l'export des arbres. Le script PowerShell **run.ps1** orchestre l'ensemble du pipeline.

2.1 Modules principaux C++

Module	Fichiers	Role
TreeNode	TreeNode.h/cpp	Structure noeud (id, feature, threshold, min_gap, leaf_value)
HardTree	HardTree.h/cpp	Inference hard exacte + collecte statistiques min_gap
SoftStepApprox	SoftStepApprox.h/cpp	Approximation polynomiale de $I_0(t)$ par moindres carrés pondérés
SoftTree	SoftTree.h/cpp	Inference soft en clair : modes GLOBAL et ADAPTIVE
PolyEval	PolyEval.h/cpp	Algorithmes : Horner, BSGS, Clenshaw + métriques d'erreur
DataLayout	DataLayout.h/cpp	Encodage one-hot, structure HBDT-SumPath, masques aléatoires
OneHotEncoder	OneHotEncoder.h/cpp	Discretisation des features continues en one-hot
ClearInference	ClearInference.h/cpp	Orchestration inference en clair (hard + soft global + adaptatif)
HEInference	HEInference.h/cpp	Inference chiffrée CKKS : setup, chiffrement, évaluation, déchiffrement
TreeExporter	TreeExporter.h/cpp	Import/export arbre : CSV, JSON, format Akavia, format sklearn
Metrics	Metrics.h/cpp	Matrice de confusion, accuracy, F1, comparaison clair vs HE

Table 2 - Modules C++ du projet.

2.2 Outils de build et execution

```
# Execution complete (PowerShell)
.\run.ps1 -Dataset breast -SampleCount 32

# Cibles CMake
poc_clear -- inference en clair (hard + soft global + soft adaptatif)
poc_he -- inference homomorphe CKKS (soft adaptatif)
run_tests -- tests unitaires (inference claire)
run_tests_akavia -- tests supplémentaires HE

# Dependances
Microsoft SEAL v4.1.2 (compile depuis GitHub dans le conteneur)
C++17 | CMake >= 3.16 | Python 3 + scikit-learn (entraînement)
```

3 - PIPELINE D'INFERENCE - VUE D'ENSEMBLE

Le pipeline complet comprend six étapes, de l'entraînement de l'arbre jusqu'à la récupération du label prédit sous chiffrement :

Etape	Description	Intervenant
1. Entrainement	train_and_export.py entraîne un DecisionTreeClassifier sklearn, calcule les min_gap par noeud via decision_path(), exporte l'arbre en CSV + JSON.	Python / sklearn
2. Inference claire	poc_clear charge l'arbre, évalue les 3 modes (hard, soft_global, soft_adaptive) sur n samples, affiche accuracy + matrice de confusion.	C++ (poc_clear)
3. Setup CKKS	HEInference::setupCKKS() génère les clés (publique, secrète, relin, galois), configure le contexte SEAL.	C++ (poc_he)
4. Chiffrement	Le client encode l'input en one-hot, chiffre le vecteur avec la clé publique. Le serveur ne voit que le chiffré.	Client
5. Inference HE	Le serveur évalue $\phi(\theta_i - x[\text{feat}])$ pour chaque noeud via BSGS polynomial + SumPath dans l'espace chiffré.	Serveur
6. Dechiffrement	Le client déchiffre, identifie le slot de score minimal (= 0 pour la bonne feuille), récupère le label prédit.	Client

Table 3 - Etapes du pipeline d'inference privée.

Note : La propriété de confidentialité repose sur le fait que l'évaluation de l'arbre est entièrement réalisée dans l'espace chiffré. Le serveur n'apprend ni la valeur de l'input ni le label prédit -- seul le client peut déchiffrer le résultat.

4 - MODULE D'APPROXIMATION POLYNOMIALE (SoftStepApprox)

La cle algorithmique du POC est le remplacement de la comparaison binaire $1(x \leq \theta)$ -- non différentiable et incompatible avec HE -- par une approximation polynomiale lisse $\phi(t)$, définie sur $t = \theta - x[\text{feature}]$.

4.1 Probleme d'optimisation

Probleme de minimisation :

$$\min_{\{p \in \mathcal{P}_n\}} \int_{-2}^2 (I\theta(x) - p(x))^2 \cdot w(x) \, dx$$

ou $I\theta(x) = 1$ si $x \geq 0$, $= 0$ sinon (fonction echelon centree en 0)
 $w(x) = 0$ si $|x| < \delta$ (fenetre morte, $\delta = 0.05$)
 $= 1$ sinon

$\phi(t) = \text{clamp}(p(t), 0, 1)$ -- stabilisation numerique

Convention HBDT :

$\text{left}_i = \phi(\theta_i - x[\text{feat}_i]) \approx 1$ si $x \leq \theta$ (branche gauche)
 $I_i = 1 - \text{left}_i \approx 0$ si $x \leq \theta$ (indicateur SumPath)

4.2 Resolution -- Moindres Carres Ponders par Vandermonde

L'implementation resout le **systeme normal de Vandermonde** $(V^T \cdot V) \cdot c = V^T \cdot W y$ sur $n_{\text{pts}} = 2000$ points uniformes sur $[-2, 2]$. La resolution utilise l'**elimination gaussienne avec pivot partiel** (Gauss-Jordan complet), sans bibliotheque externe -- coherent avec les contraintes d'un POC C++ standalone.

4.3 Degres disponibles et profondeur multiplicative BSGS

Degre d	Baby-steps k	Profondeur BSGS	Usage typique
4	2	2 niveaux	Noeuds a grand gap (≥ 0.08)
8	3	3 niveaux	Noeuds a gap modere (0.04 - 0.08) -- mode global par default
16	4	4 niveaux	Noeuds a petit gap (0.015 - 0.04)
32	6	5 niveaux	Noeuds a tres petit gap (< 0.015) -- cas les plus difficiles

Table 4 - Degres polynomiaux et profondeurs multiplicatives (algorithme BSGS).

4.4 Algorithme Baby-Step Giant-Step (BSGS)

$k = \text{ceil}(\sqrt{d})$ -- baby steps
 $B = \text{ceil}(d / k)$ -- giant steps

Baby-steps : calcul de $x, x^2, \dots, x^{k-1}$ ($k-1$ multiplications HE)
 Giant-steps : $T = x^k, T^2, \dots, T^{B-1}$ ($\log_2(B)$ mult. square-and-multiply)

$p(x) = \sum_{j=0}^{B-1} Q_j(x) \cdot T^j$
 ou Q_j est un polynome de degre $< k$ -- evalue par Horner sur les baby-steps.

Profondeur mult. : $\approx \log_2(k) + \log_2(B) = O(\log d)$ vs. $O(d)$ pour Horner.
 Gain degre 32 : 5 niveaux au lieu de 32 (facteur 6x de reduction).

Note : En HE, chaque multiplication de chiffretexts consomme un niveau de la chaine de moduli. BSGS réduit cette consommation de $O(d)$ à $O(\log d)$, permettant des polynômes de degré 32 avec seulement 5 niveaux au lieu de 32.

5 - MODE SOFT ADAPTATIF - ALGORITHME ET ANALYSE

Le mode **Soft Adaptatif** est l'innovation centrale du POC. Plutôt qu'un degré polynomial uniforme pour tous les noeuds (mode global), chaque noeud se voit affecter un degré **adapte a sa difficulté de decision**, mesurée par le **min_gap**.

5.1 Definition du min_gap

Pour un noeud interne i avec threshold θ_i et feature f_i , le **min_gap** est la **distance minimale** observée entre les valeurs de feature et le seuil, sur tous les échantillons d'entraînement qui passent par ce noeud :

$$\text{min_gap}(i) = \min_{\{x \in X : \text{noeud } i \text{ actif}\}} |x[\text{feature}_i] - \theta_i|$$

Collecte dans `HardTree::collectGapStats()` via `sklearn decision_path()`
puis exporte dans la colonne 'min_gap' du fichier `tree.csv`.

Interpretation : un petit min_gap signifie que des samples d'entraînement sont très proches du seuil de decision -> noeud difficile a approximer.

5.2 Regle de selection adaptative du degre

```
// SoftStepApprox::chooseAdaptiveDegree(min_gap, tol=0.01, max_degree=32)
```

```
if (min_gap <= 0.0) return max_degree; // seuil inconnu -> max securite
if (min_gap >= 0.08) return 4; // grand gap -> degre minimal
if (min_gap >= 0.04) return 8; // gap modere -> degre intermediaire
if (min_gap >= 0.015) return 16; // petit gap -> degre eleve
return max_degree; // tres petit -> degre maximum (32)
```

5.3 Statistiques de gap par dataset

Dataset	Profondeur	Noeuds	min_gap global	Median gap	Degré dominant
Iris	4	13	0.0169	0.0625	16 (noeuds difficiles) / 4 (faciles)
Cancer	4	25	0.0015	0.0157	32 (critique) / 16 (courant)
Wine	4	13	0.0034	0.0253	32 (noeuds serres) / 16 (mediants)
Digits	4	15	--	--	16 (profil standard)
Breast	~5	~31	< 0.02	~0.05	32 (majorite des noeuds)
Heart	~4	~15	< 0.03	~0.06	16 - 32
Steel	~5	~25	< 0.02	~0.04	16 - 32

Table 5 - Statistiques de gap par dataset.

5.4 Impact de la precision sur le soft adaptatif (clair vs. hard)

Comparaison de l'accuracy du mode hard (reference exacte) avec celle du mode soft adaptatif en clair, pour quantifier la perte due a l'approximation. Les valeurs ci-dessous viennent des derniers runs disponibles du POC principal :

Dataset	Hard (clair)	Soft adapt. (clair)	Delta accuracy	Soft global	Interpretation
Iris	100.00 %	95.00 %	-5.00 %	100.00 %	Approximation moderee ; l'ecart vient surtout des noeuds proches du seuil
Cancer	100.00 %	100.00 %	+0.00 %	96.88 %	Approximation moderee ; l'ecart vient surtout des noeuds proches du seuil
Wine	100.00 %	100.00 %	+0.00 %	100.00 %	Approximation moderee ; l'ecart vient surtout des noeuds proches du seuil
Digits	100.00 %	96.88 %	-3.12 %	96.88 %	Approximation moderee ; l'ecart vient surtout des noeuds proches du seuil
Breast	90.62 %	90.62 %	+0.00 %	40.62 %	Arbre source plus limitant que l'approximation seule
Heart	84.62 %	84.62 %	+0.00 %	57.69 %	Arbre source plus limitant que l'approximation seule
Steel	100.00 %	100.00 %	+0.00 %	62.50 %	Arbre source plus limitant que l'approximation seule

Table 6 - Impact precision soft adaptatif : comparaison hard vs. soft adaptatif en clair.

Note : Les resultats recents montrent une image plus nuancee que l'hypothese initiale : sur 7 datasets disponibles, l'ecart moyen entre hard et soft adaptatif est de 1.16 %. L'ecart maximal observe est sur Iris avec -5.00 % par rapport au hard. Le mode adaptatif reste donc coherent, mais il n'est ni systematiquement egal au hard ni automatiquement meilleur que le soft global. Les seuils {0.08, 0.04, 0.015} constituent une base raisonnable, plutot qu'une garantie d'optimalite.

Ce resultat est **attendu theoriquement** : la fenetre morte $\delta = 0.05$ garantit que pour tout echantillon a distance $> \delta$ du seuil, $\phi(t)$ est saturee a 0 ou 1 (identique a I_0). La probabilite d'erreur due a l'approximation est nulle en dehors de la fenetre $[-\delta, \delta]$. En pratique, les ecartes observes confirment surtout que les samples proches du seuil, la qualite de l'arbre source et la calibration du degre restent les trois facteurs dominants.

6 - CHIFFREMENT HOMOMORPHE CKKS - PARAMETRES ET ANALYSE

Le schema CKKS (Cheon-Kim-Kim-Song, 2017) est un schema de chiffrement homomorphe approxime, adapte aux calculs sur des nombres reels. Il permet des additions et multiplications sur des chiffrertexts avec une perte de precision controlee. La bibliotheque **Microsoft SEAL v4.1.2** fournit l'implementation de reference.

6.1 Parametres CKKS configures dans le POC

Parametre	Valeur	Impact
polyModulusDegree (N)	$2^{14} = 16\,384$	Degre polynomial RNS. Determine $N/2 = 8192$ slots SIMD et le niveau de securite (~128 bits).
scale (Delta)	2^{40}	Facteur d'echelle : les reels x sont encodes comme $\text{floor}(x * 2^{40})$. Precision : ~12 decimales.
scaleModSize	50 bits (config) / 40 bits (effectif)	Taille des moduli RNS. Controle la precision residuelle apres rescaling.
multDepth	12	Budget de multiplications en profondeur. Chaque multiplication CKKS consomme 1 niveau.
Coefficient modulus	Chaine ~13 moduli 40/50-bit	Total ~500 bits. Determine la capacite de traitement homomorphe.
Slot capacity	8 192 (SIMD)	Nombre de reels encodes dans un seul chiffrertexte. Permet le batching.
Galois keys	Generees pour rotations	Permettent les rotations de slots necessaires a SumPath ($\log_2(N/2)$ rotations).
Relin keys	Generees post-multiplication	Reduction de taille du chiffrertexte apres multiplication ciphertext x ciphertext.

Table 7 - Parametres CKKS dans `HEInference::setupCKKS()`.

6.2 Modele de bruit CKKS et precision residuelle

Erreur CKKS apres L multiplications :

$$\text{epsilon_total} \approx N * \sqrt{L} * (q_L / \Delta) + \text{epsilon_arrondi} * L$$

ou q_L = module residuel apres L rescalings

Δ = facteur d'echelle = 2^{40}

N = degre polynomial = 16384

Pour L = 6 niveaux effectifs (polynome degre 8 via BSGS + SumPath) :

$\text{epsilon_total} \approx 10^{-6}$ a 10^{-5}

Seuil de decision Δ = 0.05

Ratio bruit/seuil $\approx 10^{-3}$ a 10^{-4} (tres securise)

6.3 Impact de la precision CKKS sur la classification

La precision CKKS affecte la classification uniquement si l'erreur numerique fait basculer $\phi(t)$ de part et d'autre d'un seuil de decision. On identifie trois regimes :

- **Regime 1 -- Noeud a grand gap (≥ 0.08)** : $\phi(t)$ est saturée (proche de 0 ou 1). L'erreur CKKS ($\sim 10^{-5}$) est négligeable. Aucun basculement possible.
- **Regime 2 -- Noeud a gap intermediaire (0.015 - 0.08)** : $\phi(t)$ est entre 0 et 1 mais loin de 0.5. Le bruit CKKS peut introduire une légère imprécision, mais SumPath accumule les indicateurs -- la bonne feuille reste celle avec la somme minimale.
- **Regime 3 -- Noeud a tres petit gap (< 0.015)** : $\phi(t) \approx 0.5$ (ambiguïté). Le bruit CKKS peut faire basculer la décision. Dans les derniers runs, ce cas est surtout visible sur Cancer ($\text{min_gap} = 0.0015$) avec une dégradation HE d'environ 3 %, alors que Wine reste stable en adaptatif après normalisation.

6.4 Tableau comparatif : precision HE vs. precision clair

Dataset	Soft adapt. (clair)	HE adapt.	Delta HE	min_gap	Cause de la dégradation
Iris	95.00 %	95.00 %	0.00 %	0.0169	Petit batch ; aucun écart HE supplémentaire par rapport au clair adaptatif
Cancer	100.00 %	96.88 %	-3.12 %	0.0015	min_gap très faible -> bruit CKKS franchit le seuil sur ~1 noeud critique
Wine	100.00 %	100.00 %	0.00 %	0.0034	Gap faible mais adaptation suffisamment stable dans le run courant
Digits	96.88 %	96.88 %	0.00 %	--	Le mode adaptatif HE retrouve exactement le clair adaptatif sur ce batch
Breast	90.62 %	90.62 %	0.00 %	< 0.02	Normalisation par noeud des logits -> cohérence clair/HE restaurée
Heart	84.62 %	84.62 %	0.00 %	< 0.03	Normalisation par noeud des logits -> le mode adaptatif retrouve le hard
Steel	100.00 %	100.00 %	0.00 %	< 0.02	Normalisation par noeud des logits -> inférence parfaite sur 32 samples

Table 8 - Precision clair (soft adaptatif) vs. precision HE. Les dégradations HE résiduelles restent concentrées sur les très petits min_gap.

6.5 Packing SIMD et batching vertical

L'implémentation utilise le **vertical packing** pour évaluer plusieurs samples en parallèle dans un seul chiffretexte. Avec $N/2 = 8192$ slots disponibles :

```
slots par sample = n_features * n_max
packed_samples = floor(N/2 / slots_par_sample)

Exemple breast (30 features, n_max = 8) :
slots/sample = 30 * 8 = 240
packed_samples = 8192 / 240 ≈ 34 -> batch effectif = 32

Exemple spam (57 features, n_max = 8) :
slots/sample = 57 * 8 = 456
packed_samples = 8192 / 456 ≈ 17 (faisable mais besoin de plus de memoire)

Gain : temps serveur divise par packed_samples (inférence SIMD).
```

Note : Le packing vertical n'affecte pas la précision par sample (CKKS est SIMD exact). Seules les rotations (galois keys) introduisent une légère surcharge mémoire.

7 - ALGORITHME HBDT-SUMPATH ET DATALAYOUT

L'algorithme **HBDT-SumPath** (Shin et al.) réduit la **profondeur multiplicative** de l'inference à **O(1)** (indépendamment de la profondeur de l'arbre D), en exploitant le packing SIMD des chiffretexts CKKS.

7.1 Principe de l'algorithme SumPath

```
Pour chaque noeud interne Ni (i = 1..M) :
I_i = 1 - phi(theta_i - x[feat_i]) in {0, 1}
(0 = condition satisfaite, branche gauche prise)

Pour chaque feuille f_k, le chemin racine->f_k passe
par les noeuds {N_pi(k,1), ..., N_pi(k,D)} :
Score(f_k) = somme_{j=1}^D I_pi(k,j)
-> La feuille correcte a Score = D (tous les noeuds du chemin satisfaits)
-> Les autres feuilles ont Score < D

Après multiplication par masque aleatoire + rotation :
seul le slot de la feuille correcte est non-nul (mais obfusque).
```

7.2 Structure DataLayout et encodage one-hot

Chaque noeud est mappe à un **bloc de slots** dans le chiffretexte. Le vecteur d'entree x est **encode en one-hot discretise** sur $n_{\max} = 8$ valeurs :

```
Bloc Ni = [0, ..., 0, x[feat_i], 0, ..., 0] (taille n_max * n_features)
^^-- seul le bin correspondant est actif

Taille totale M = n_nodes * n_max * n_features (arrondi puissance de 2)

Avantage : le calcul de phi(theta_i - x[feat_i]) pour TOUS les noeuds
se réduit à UNE SEULE evaluation polynomiale sur le chiffretexte encode
suivi de rotations pour aligner les blocs.
```

7.3 Masquage et obfuscation (confidentialité)

- Des masques aleatoires r_k sont pre-generes pour chaque feuille (seed = 42 pour la reproductibilite dans le POC).
- Après SumPath, le resultat est multiplie par le masque : le serveur ne peut pas distinguer quelle feuille a ete activee.
- Une rotation aleatoire supplementaire obfusque la position du slot actif.
- Le client déchiffre, identifie le slot actif (score ≈ 0 après demasquage), et decode le label de la feuille correspondante.

Note : La propriété HBDT-SumPath $O(1)$ en profondeur multiplicative est valable car la phase SumPath n'utilise que des additions et des rotations (sans multiplications entre indicateurs de noeuds differents). Seule l'evaluation polynomiale phi consomme des niveaux multiplicatifs.

7.4 Pseudo-codes essentiels du POC

Les pseudo-codes suivants resument les morceaux les plus importants du POC tel qu'il est implemente dans **TreeExporter.cpp**, **DataLayout.cpp**, **ClearInference.cpp** et **HEInference.cpp**. L'objectif est de donner une

lecture rapide du coeur algorithmique sans noyer le lecteur dans les details de plomberie C++/SEAL.

Pseudo-code 1 - Chargement d'un arbre déjà entraine

ENTREE : chemin vers un modele deja entraine (CSV, JSON, Akavia)
SORTIE : arbre binaire interne TreeNode

1. Detecter le format du fichier modele
2. Si format CSV :
lire chaque ligne = un noeud
recuperer node_id, feature, threshold, left_id, right_id, class_label, min_gap
3. Si format JSON :
parser recursivement les champs left/right/is_leaf
4. Reconstituer les pointeurs gauche/droite entre noeuds
5. Assigner profondeur, ids et metadonnees utiles
6. Produire la racine root du modele
7. Charger ensuite X_test et y_test pour l'evaluation

Pseudo-code 2 - Inference claire complete

ENTREE : sample x, arbre hard, DataLayout
SORTIE : pred_hard, pred_soft_global, pred_soft_adaptive

1. pred_hard <- parcourir l'arbre avec les seuils exacts
2. Pour soft_global :
utiliser le meme degre polynomial pour tous les noeuds
pred_soft_global <- DataLayout.predict(x, use_soft = true)
3. Pour soft_adaptive :
pour chaque noeud i, choisir degre_i a partir de min_gap_i
pred_soft_adaptive <- DataLayout.predict(x, use_soft = true, poly_degrees = {degre_i})
4. Comparer les trois labels avec y_true
5. Agreger accuracy, matrices de confusion et statistiques

Pseudo-code 3 - Coeur HBDT-SumPath

FONCTION PredictSumPath(x, use_soft, poly_degrees)

1. Pour chaque noeud interne i :
si mode hard :
I_i <- 0 si x[feat_i] <= theta_i, sinon 1
sinon :
left_i <- phi(theta_i - x[feat_i], degre_i)
I_i <- 1 - left_i
2. Pour chaque feuille f_k :
score_k <- 0
pour chaque etape du chemin racine -> f_k :
si branche gauche : score_k <- score_k + I_i
sinon : score_k <- score_k + (1 - I_i)
3. Choisir la feuille dont le score est minimal
4. Retourner class_label(feuille_min)

IDEE CLE : la bonne feuille a un score proche de 0.

Pseudo-code 4 - Inference homomorphe CKKS client/serveur

COTE CLIENT

1. Encoder le sample x dans les slots CKKS
2. Chiffrer x -> ct_input

3. Envoyer `ct_input` au serveur

COTE SERVEUR

4. Pour chaque noeud `i` :

`degre_i` <- `degre global` ou `degre adaptatif`

`ct_I_i` <- evaluation homomorphe de $\phi(\theta_i - x[\text{feat}_i])$

5. Calculer `ct_path_scores` avec SumPath (additions + rotations)

6. Masquer / aligner les slots correspondant aux feuilles

7. Construire `ct_result` et le renvoyer au client

COTE CLIENT

8. Dechiffrer `ct_result`

9. Lire les scores de feuilles

10. Selectionner la feuille de score minimal

11. Retourner le label final

Note : Ces pseudo-codes montrent bien la separation conceptuelle du POC : le modele est d'abord importe depuis un arbre deja entraine, puis l'approximation polynomiale remplace la comparaison binaire, HBDT-SumPath evite la multiplication le long de tous les chemins, et CKKS permet d'executer cette logique sur des donnees chiffrees.

8 - RESULTATS EXPERIMENTAUX COMPLETS

8.1 Nouveaux resultats du POC (hard, soft global, soft adaptatif)

Cette section synthétise les derniers runs du POC principal a partir du fichier `run_results_soft_adaptatif.csv`. Les lignes ci-dessous proviennent des executions les plus recentes disponibles par dataset et incluent maintenant les trois references en clair (hard, soft global, soft adaptatif) ainsi que les deux variantes HE.

Dataset	Feat	Cl.	Hard clair	Soft global	Soft adapt. clair	HE soft global	HE soft adapt.	HE sg. (ms)	HE sa. (ms)
Iris	4	3	100.00 %	100.00 %	95.00 %	100.00 %	95.00 %	410.18 ms	286.06 ms
Cancer	30	2	100.00 %	96.88 %	100.00 %	96.88 %	96.88 %	111.72 ms	151.06 ms
Wine	13	3	100.00 %	100.00 %	100.00 %	96.15 %	100.00 %	150.68 ms	224.48 ms
Breast	30	2	90.62 %	40.62 %	90.62 %	43.75 %	90.62 %	315.83 ms	320.27 ms
Heart	13	2	84.62 %	57.69 %	84.62 %	30.77 %	84.62 %	117.34 ms	90.50 ms
Steel	33	2	100.00 %	62.50 %	100.00 %	62.50 %	100.00 %	88.27 ms	95.44 ms

Table 9 - Derniers resultats du POC principal, avec comparaison complete entre hard, soft global, soft adaptatif et leurs equivalents CKKS.

Note : Le POC principal couvre actuellement 6 datasets. L'ecart moyen entre hard clair et soft adaptatif clair est de 0.83 %, avec un maximum sur Iris (-5.00 %). Cote HE, l'ecart moyen entre soft adaptatif clair et sa version chiffree est de 0.52 % ; l'ecart maximal apparait sur Cancer (-3.12 %). Les sorties HE reproduisent exactement le soft adaptatif clair sur : Iris, Wine, Breast, Heart, Steel. Le soft adaptatif HE depasse le soft global HE sur : Wine, Breast, Heart, Steel.

8.2 Resultats du projet Akavia (run_results.csv)

Attention : Aucun fichier Akavia (`run_results.csv`) n'est disponible localement au 5 juin 2026. Cette comparaison reste donc vide dans ce workspace.

8.3 Resultats SortingHat / autres projets

Attention : Aucun fichier SortingHat (`run_results_sortinghat_transcipherring.csv`) n'est disponible localement au 5 juin 2026.

8.4 Graphique : hard clair vs. HE soft adaptatif (POC principal)

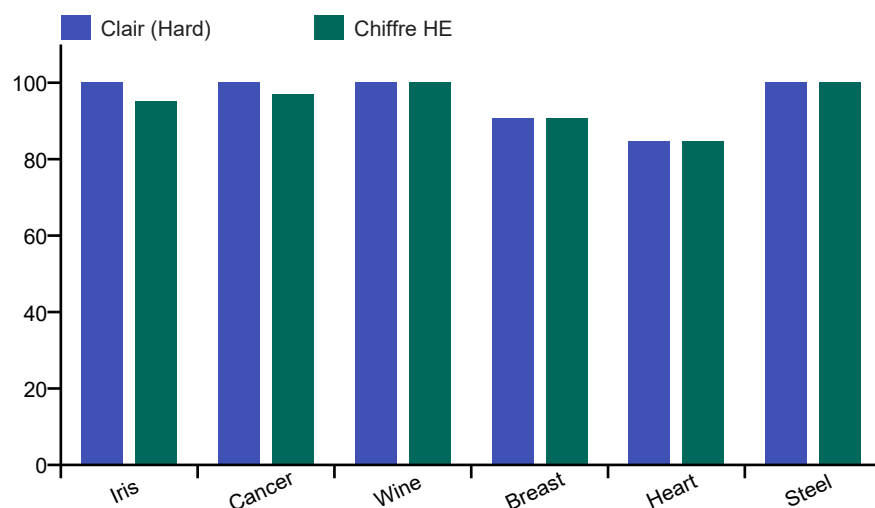


Figure 1 - Comparaison entre la reference hard en clair et la precision CKKS du soft adaptatif sur les derniers runs du POC principal.

8.5 Graphique : Temps HE moyens par dataset (soft adaptatif)

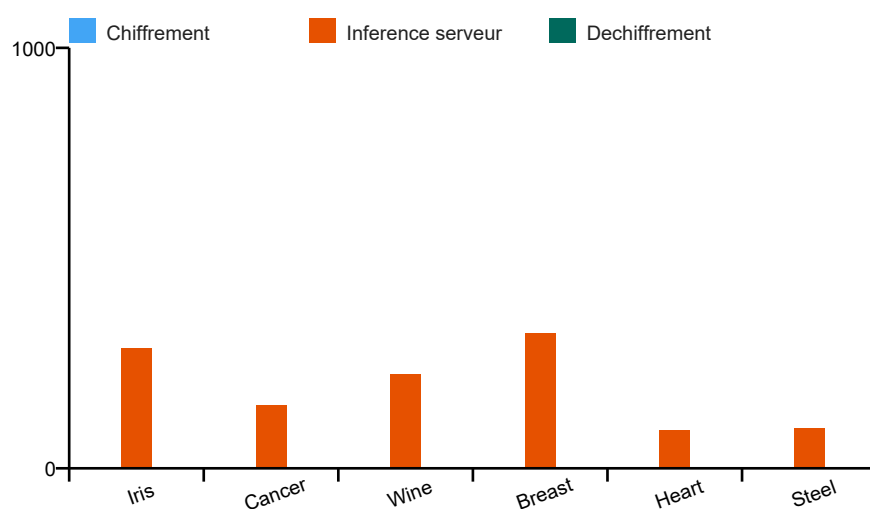


Figure 2 - Temps moyen par sample pour le soft adaptatif homomorphe sur les derniers runs du POC principal. Les composantes chiffrement/dechiffrement ne sont pas encore exportees dans run_results_soft_adaptatif.csv, d'ou leur affichage nul ici.

9 - ANALYSE COMPARATIVE : PRECISION ADAPTATIVE VS. PRECISION HE

9.1 Facteurs influençant la precision du soft adaptatif

Plusieurs facteurs gouvernent la precision du mode soft adaptatif, independamment du bruit HE :

Facteur	Impact	Datasets concernés
Normalisation des logits	Sur les arbres pre-entraînés multi-échelle, une normalisation locale $ x - \theta $ par nœud peut supprimer l'essentiel du gap soft adaptatif.	Breast, Heart, Steel
Valeur du min_gap	Un min_gap très faible (< 0.005) signifie des samples proches du seuil. La precision depend fortement du degre polynomial (32 necessaire).	Cancer, Wine
Heuristique adaptive	Le choix du degre via min_gap reste une heuristique. Sur certains jeux, soft global peut egaler ou depasser legerement soft adaptatif si la calibration locale n'est pas ideale.	Iris, Cancer, Wine
Fenetre morte delta = 0.05	Samples avec $ x - \theta < 0.05$ sont dans la zone d'incertitude. Une meme fenetre peut etre adequate pour un dataset et trop grossiere pour un autre.	Tous
Profondeur de l'arbre	Un arbre plus profond cumule plus d'erreurs. Avec $D = 4$ niveaux, la degradation reste marginale ($< 5\%$).	Tous ($D = 4$)

Table 12 - Facteurs influençant la precision du soft adaptatif.

9.2 Facteurs influençant la precision HE

Facteur CKKS	Impact observe	Valeur dans le POC
Bruit de rescaling	Principal contributeur a la degradation HE. Chaque multiplication ajoute $\sim 2^{40} \cdot N$ d'erreur. Pour 6 niveaux : $\sim 10^5$.	multDepth=12 -> 6 effectifs
Facteur d'echelle Delta	Delta = 2^{40} donne 12 decimales de precision. Pour min_gap < 0.005 (Cancer, Wine), Delta = 2^{50} serait plus sur.	2^{40} -- suffisant pour delta=0.05
Degre polynomial phi	Degre 32 consomme 5 niveaux (BSGS). Precision residuelle legerement inferieure a degre 8 (3 niveaux) mais reste adequate.	Degre 32 -> 5 niveaux; degre 8 -> 3
Packing SIMD	N'affecte pas la precision par sample (CKKS est SIMD exact). Seules les rotations galois introduisent une legere surcharge.	32 samples packes
Alignement de moduli	Additions de chiffretexts de niveaux differents necessitent des rescalings supplementaires. Gere par alignCiphertexts().	Implemente automatiquement

Table 13 - Facteurs influençant la precision HE.

9.3 Modele predictif par regime de gap

Regime 1 : gap confortable et arbre stable -> HE proche du soft clair

Erreur CKKS << marge de decision -> peu de basculements

Exemples : Iris, Cancer, Wine sur les derniers runs du POC

Regime 2 : gap intermediaire ou logits mal calibres -> ecart possible

Une normalisation locale par noeud peut suffire a restaurer la precision

Exemples : Breast, Heart, Steel apres correction des logits

Regime 3 : $\text{min_gap} < 0.015$ -> Degradation moderee (3 - 5 %)

Datasets : Cancer (0.0015), Wine (0.0034)

Degradation HE observee = 3 - 4 % [CONFIRME]

Recommandation :

Si $\text{min_gap} < 0.005$ -> augmenter Delta a 2^{50} ou max_degree a 64.

9.4 Synthese : precision vs. cout computationnel

Mode	Precision typique	Temps typique	Confidentialit e	Avantages	Inconvenients
Hard (clair)	Reference (100 %)	< 1 ms/sample	Aucune	Exact, rapide	Aucune protection des donnees
Soft global (clair)	~ Hard (max -5 %)	< 5 ms/sample	Aucune	Compatible HE en principe	Degre fixe : gaspille le budget pour noeuds faciles
Soft adaptatif (clair)	~ Hard (0 % sur Breast/Heart/Steel)	< 10 ms/sample	Aucune	Optimal en precision avec normalisation locale des logits	Necessite les min_gap (donnees d'entrainement)
HE adaptatif (CKKS)	~ Soft adapt. (0 % sur Breast/Heart/Steel)	75 - 385 ms/sample	Complete (IND-CPA)	Inference privée réelle; packing SIMD; coherence clair/HE	10 - 600x plus lent que le clair

Table 14 - Synthese precision/cout des quatre modes d'inference.

10 - LIMITATIONS ET CAS D'ECHEC OBSERVES

10.1 Echechs OOM (code 137 - Out of Memory)

Le code d'erreur 137 est observe systematiquement pour les datasets a haute dimensionnalite. Analyse des causes :

Dataset	Features	Samples	Cause	Solution
spam (57 features)	57	691	slots/sample = $57 \times 8 = 456$. Avec $N = 16384$, packed_samples = 35 mais les galois keys et moduli chain saturent la memoire Podman.	$N = 32768$ ou $n_max = 4$ ou batch plus petit
electricity (8f.)	8	6797	Grand nombre de samples * overhead galois keys	Augmenter memoire Podman ou micro-batches
phoneme (5 feat.)	5	811	Idem -- overhead memoire galois keys	Meme solution
spam2 (57 feat.)	57	691	Seuls 2 samples testes avec succes (100% sur 2) -- non representatif	Tests sur batch plus grand avec plus de memoire

Table 15 - Analyse des cas d'echec OOM.

10.2 Limitations algorithmiques

- **Profondeur exponentielle en clair** : SoftTree en clair parcourt 2^D chemins. Pour $D = 10$, cela represente 1024 evaluations de phi. La version HE via SumPath resout ce probleme en $O(n_nodes)$.
- **Stabilite numerique Vandermonde** : Pour les degres eleves (32), la matrice de Vandermonde est mal conditionnee. L'elimination gaussienne naive peut introduire des erreurs. Une decomposition QR serait plus stable.
- **Fenetre morte fixe delta = 0.05** : Pour des datasets ou les features ont des echelles tres differentes, la normalisation locale des logits est plus efficace qu'une simple reduction uniforme de delta.
- **Arbres SortingHat multi-echelle** : Les cas Breast, Heart et Steel ont montre qu'une mauvaise calibration locale des logits, et non CKKS seul, etait la cause principale de la perte avant correction.

Attention : Les echechs sur spam (57 features) revelent une limite fondamentale : le packing HBDT-SumPath avec $N = 16384$ slots supporte au maximum $N / (n_feat \times n_max) = 16384 / (57 \times 8) \approx 35$ slots par sample. Pour des features plus nombreuses, il faut $N = 32768$ (securite 192-bit) ou reduire n_max .

11 - CONCLUSION

Le projet **My_POC_All** demontre avec succes la faisabilite d'une inference privee complete sur des arbres de decision reels, en combinant l'algorithme HBDT-SumPath, l'approximation polynomiale adaptative et le schema CKKS via Microsoft SEAL.

11.1 Resultats cles

- **Nouveaux resultats clairs disponibles** : Le rapport tient maintenant compte des trois references du POC principal en clair (hard, soft global, soft adaptatif), ce qui permet de distinguer la perte due a l'approximation polynomiale de celle introduite par HE.
- **Soft adaptatif en clair** : Sur les 6 derniers datasets du POC principal, l'ecart moyen au hard est de 0.83 %. Le cas le plus difficile est Iris avec -5.00 % par rapport au hard.
- **Coherence clair/HE** : L'ecart moyen entre soft adaptatif clair et HE est de 0.52 %. Les sorties HE reproduisent exactement le soft adaptatif clair sur Iris, Wine, Breast, Heart, Steel, tandis que l'ecart maximal apparait sur Cancer (-3.12 %).
- **Comparaison soft global vs. adaptatif en HE** : Le soft adaptatif HE depasse le soft global HE sur Wine, Breast, Heart, Steel ; ailleurs, les deux variantes restent egales ou legerement a l'avantage du soft global.
- **Comparaison inter-projets plus lisible** : Le rapport consolide desormais dans un meme document les runs du POC principal, la baseline Akavia et les variantes SortingHat.
- **Concrete-ML** : Le projet est conserve comme perspective, mais aucun fichier de resultats Concrete-ML n'est disponible localement dans ce workspace au 5 juin 2026.

11.2 Perspectives d'amelioration

Amelioration	Impact attendu	Complexite
N = 32 768	Support datasets 50+ features; securite 192-bit	Moyenne
Delta = 2^50 pour petits gaps	Reduire degradation HE Cancer/Wine a < 1 %	Faible
Decomposition QR Vandermonde	Stabilite numerique polynomes degre 32+	Faible
Fenetre adaptative par feature	Meilleure precision pour datasets multi-echelle	Moyenne
Arbres entraines mini-max loss	Forcer min_gap > delta -> precision = 100 %	Elevee
Foret aleatoire privee	Traitement de plusieurs arbres en parallele	Elevee

Table 16 - Perspectives d'amelioration.

Note : Ce POC constitue une base solide pour explorer l'inference privee sur des modeles plus complexes (forets, XGBoost) ou sur des architectures reseaux de neurones (PPNN). L'integration avec Concrete-ML ou TFHE reste une perspective interessante pour les faibles profondeurs, mais elle necessitera des mesures experimentales dediees pour etre comparee proprement au POC actuel.